DESARROLLO DE APLICACIONES MÓVILES

ACTIVIDAD AUTÓNOMA
SESIÓN 5 · UNIDAD 1

Manual de arquitectura de PharmaMobil

Responsabilidades por capa y pruebas de flujo

Reto 02 · Clean + MVVM en Kotlin Multiplatform

Apellidos y nombres: Acevedo, [completar nombres]
Código: [completar]
Fecha de entrega: [completar]

Curso: Desarrollo de Aplicaciones Móviles
Unidad 1: Fundamentos de Kotlin Multiplatform y UI con Compose Multiplatform
Modalidad: trabajo independiente

Repositorio:
https://github.com/miguelon05/PharmaMobile

Rama local de la actividad: codex/actividad-autonoma-s5
Base revisada: 1565816 · GUIA PRACTICA 08/09

Documento en negro sobre fondo blanco. Las cajas de evidencia indican las capturas reales que deben insertarse; no representan capturas de una ejecución.

Índice

1. Mapa de capas 3

2. Tabla de responsabilidades 4

3. Decisiones justificadas 9

4. Pruebas de flujo 10

5. Bitácora de dificultades 13

Objetivo y alcance

Documentar la arquitectura Clean + MVVM del proyecto KMP, relacionando cada componente con su capa y responsabilidad, y verificar mediante commonTest el recorrido de los estados de la interfaz.

El manual describe el código presente en shared, incluidos los componentes heredados, las funciones Compose y los puntos de arranque por plataforma. Las clases de pruebas se distinguen de las clases de producción. Pedidos conserva modelos de dominio y un destino de navegación, pero su pantalla operativa todavía no está implementada.

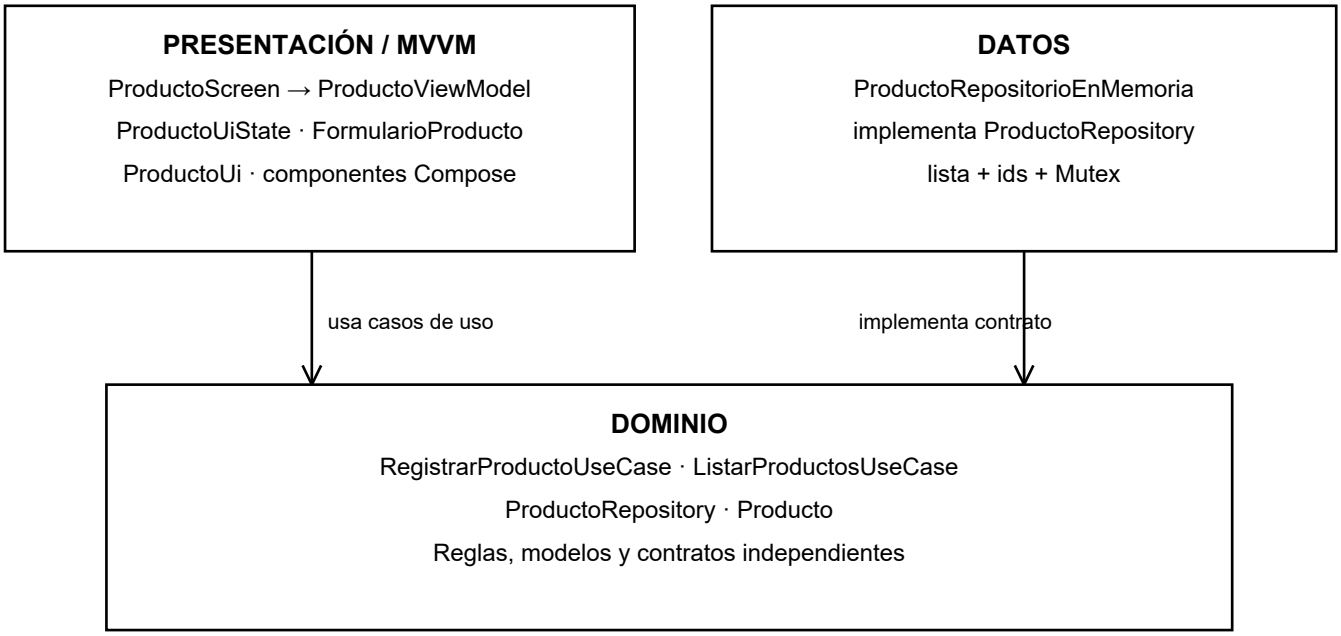
La guía se usa como especificación del trabajo. Las afirmaciones sobre implementación y resultados se contrastaron con el repositorio local y los registros generados durante esta actividad. El tutorial anterior contiene referencias históricas que ya no describen toda la versión actual.

Estado de la entrega

Pruebas Android: 53 ejecutadas, 0 fallos, 0 errores; cuatro corresponden al nuevo ProductoFlujoTest. iOS: compilación de las pruebas completada, ejecución omitida por el host Windows. Capturas reales, datos personales, publicación de la rama y envío al aula: pendientes de completar.

1. Mapa de capas

AppModule (composición): ensambla presentación, dominio y datos.



Las flechas señalan dependencias de código; el dominio no depende del exterior.

Figura 1. Dependencias de Producto en PharmaMobil. Elaboración propia, a partir del código del repositorio.

La presentación solicita operaciones a los casos de uso; estos dependen del contrato ProductoRepository y del modelo Producto. La implementación en memoria depende del contrato que implementa. AppModule es el punto externo de composición: importa las clases y registra sus dependencias en Koin; ninguna clase del dominio necesita conocerlo.

Recorrido de una carga

ProductoScreen observa uiState con collectAsStateWithLifecycle. ProductoViewModel inicia cargarProductos(), emite Cargando y llama a ListarProductosUseCase. El contrato entrega una lista o un fallo; el ViewModel convierte los modelos con aUi() y publica SinProductos, ConProductos o Error. El resultado vuelve hacia la pantalla como datos, sin crear una dependencia del dominio hacia la UI.

Recorrido de un registro

Los cambios de los campos actualizan FormularioProducto. Al registrar, RegistrarProductoUseCase valida nombre, precio y stock antes de invocar el repositorio. Si el precio es cero, devuelve ProductoInvalidoException con ErroresDeProducto; el ViewModel asigna precioError y conserva la fase del inventario. Si el registro es válido, limpia el formulario, anuncia el éxito y recarga el inventario.

Extensión a clientes y plataformas

Clientes sigue el mismo patrón mediante ClienteViewModel, RegistrarClienteUseCase, ListarClientesUseCase y ClienteRepository. Android e iOS aportan platformModule; initKoin incorpora los módulos comunes y el específico. MainViewController integra Compose en iOS. Los modelos Pedido y DetallePedido permanecen dentro del dominio.

2. Tabla de responsabilidades

Rutas relativas a shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil, salvo indicación. Se incluyen las clases, interfaces, tipos anidados y componentes funcionales relevantes: ProductoScreen, AppModule y PharmaMobilTheme son funciones o archivos de composición, no clases Kotlin.

Clase / componente	Capa	Responsabilidad en una frase	Lo que tiene prohibido hacer	Qué se rompería si desapareciera
Producto	Dominio	Representa producto; exige datos válidos y calcula reposición.	Acceder a UI o persistencia.	Faltan entidad y regla de stock mínimo.
Cliente	Dominio	Representa identidad y contacto con invariantes básicas.	Renderizar o enviar datos a red.	Pedidos y registro pierden su cliente tipado.
Pedido	Dominio	Agrupa cliente, detalles y estado del pedido.	Orquestar navegación o almacenamiento.	Desaparece el agregado del pedido.
DetallePedido	Dominio	Exige cantidad positiva y calcula subtotal.	Consultar repositorios o dibujar precios.	Se pierde la línea y su cálculo válido.
EstadoPedido	Dominio	Limita los estados posibles del pedido.	Cambiar pantallas o guardar pedidos.	El estado deja de estar tipado y acotado.
EstadoPedido.Pendiente	Dominio	Representa pedido por atender.	Ejecutar por sí mismo la transición.	Falta esta variante en el ciclo del pedido.
EstadoPedido.Procesando	Dominio	Representa pedido en preparación.	Ejecutar por sí mismo la transición.	Falta esta variante en el ciclo del pedido.
EstadoPedido.Entregado	Dominio	Representa pedido entregado.	Ejecutar por sí mismo la transición.	Falta esta variante en el ciclo del pedido.
EstadoPedido.Rechazado	Dominio	Representa rechazo con motivo.	Ejecutar por sí mismo la transición.	Falta esta variante en el ciclo del pedido.
ProductoRepository	Dominio	Define registrar y listar productos como suspend.	Conocer HTTP, Compose o Koin.	Casos de uso pierden el contrato sustituible.
ClienteRepository	Dominio	Define registrar y listar clientes como suspend.	Depender del repositorio concreto.	Se acoplarían clientes y almacenamiento.
RegistrarProductoUseCase	Dominio	Valida campos y registra un producto válido.	Manipular widgets o resolver Koin.	No hay entrada central de registro validado.
ListarProductosUseCase	Dominio	Obtiene la lista mediante el contrato y devuelve Result.	Formatear tarjetas o instanciar datos.	ViewModel pierde la operación de consulta.
ErroresDeProducto	Dominio	Agrupa errores de validación por campo.	Dibujar o almacenar entidades.	No llegan errores independientes al formulario.
ProductoInvalidoException	Dominio	Transporta errores de campos en Result.failure.	Actualizar uiState.	Se pierde la distinción de validación y fallo externo.
RegistrarClienteUseCase	Dominio	Valida campos y registra un cliente válido.	Manipular widgets o resolver Koin.	No hay entrada central de registro validado.

Clase / componente	Capa	Responsabilidad en una frase	Lo que tiene prohibido hacer	Qué se rompería si desapareciera
ListarClientesUseCase	Dominio	Obtiene la lista mediante el contrato y devuelve Result.	Formatear tarjetas o instanciar datos.	ViewModel pierde la operación de consulta.
ErroresDeCliente	Dominio	Agrupa errores de validación por campo.	Dibujar o almacenar entidades.	No llegan errores independientes al formulario.
ClienteInvalidoException	Dominio	Transporta errores de campos en Result.failure.	Actualizar uiState.	Se pierde la distinción de validación y fallo externo.
resultadoDe (función)	Dominio	Convierte fallos a Result y propaga cancelación.	Tratar cancelación como error de negocio.	Casos de uso pierden su manejo común de fallos.
ProductoRepositorioEnMemoria	Datos	Mantiene lista e ids en memoria con exclusión mutua.	Ser dueño del formulario o navegar.	El contrato queda sin almacenamiento de ejecución.
ClienteRepositorioEnMemoria	Datos	Mantiene lista e ids en memoria con exclusión mutua.	Ser dueño del formulario o navegar.	El contrato queda sin almacenamiento de ejecución.
ProductoViewModel	Presentación	Orquesta casos de uso y expone StateFlow de UI.	Acceder a la implementación de datos.	La pantalla pierde acciones y estado persistente en memoria.
ProductoUiState	Presentación	Reúne fase, formulario, registro y confirmación.	Ejecutar consultas o validación de negocio.	No hay estado único coherente para renderizar.
FormularioProducto	Presentación	Conserva entradas y mensajes de cada campo.	Registrar o definir reglas de negocio.	Se pierde el estado del formulario.
ProductoUi	Presentación	Entrega datos preparados para mostrar.	Modificar entidades o almacenar datos.	El formato volvería a mezclarse con la pantalla.
ProductoUiState.Fase	Presentación	Define fases excluyentes mediante sealed interface.	Consultar repositorios.	Se pierde exhaustividad del when de pantalla.
ProductoUiState.Fase.Cargando	Presentación	Representa consulta en curso.	Ejecutar la carga o el reintento.	No puede representarse esta situación de UI.
ProductoUiState.Fase.SinProductos	Presentación	Representa lista vacía.	Ejecutar la carga o el reintento.	No puede representarse esta situación de UI.
ProductoUiState.Fase.ConProductos	Presentación	Representa lista preparada para UI.	Ejecutar la carga o el reintento.	No puede representarse esta situación de UI.
ProductoUiState.Fase.Error	Presentación	Representa fallo con mensaje.	Ejecutar la carga o el reintento.	No puede representarse esta situación de UI.
ProductoScreen (función)	Presentación	Observa el estado y delega eventos al ViewModel.	Validar negocio o crear repositorios.	No se puede operar visualmente este módulo.
ClienteViewModel	Presentación	Orquesta casos de uso y expone StateFlow de UI.	Acceder a la implementación de datos.	La pantalla pierde acciones y estado persistente en memoria.

Clase / componente	Capa	Responsabilidad en una frase	Lo que tiene prohibido hacer	Qué se rompería si desapareciera
ClienteUiState	Presentación	Reúne fase, formulario, registro y confirmación.	Ejecutar consultas o validación de negocio.	No hay estado único coherente para renderizar.
FormularioCliente	Presentación	Conserva entradas y mensajes de cada campo.	Registrar o definir reglas de negocio.	Se pierde el estado del formulario.
ClienteUi	Presentación	Entrega datos preparados para mostrar.	Modificar entidades o almacenar datos.	El formato volvería a mezclarse con la pantalla.
ClienteUiState.Fase	Presentación	Define fases excluyentes mediante sealed interface.	Consultar repositorios.	Se pierde exhaustividad del when de pantalla.
ClienteUiState.Fase.Cargando	Presentación	Representa consulta en curso.	Ejecutar la carga o el reintento.	No puede representarse esta situación de UI.
ClienteUiState.Fase.SinClientes	Presentación	Representa lista vacía.	Ejecutar la carga o el reintento.	No puede representarse esta situación de UI.
ClienteUiState.Fase.ConClientes	Presentación	Representa lista preparada para UI.	Ejecutar la carga o el reintento.	No puede representarse esta situación de UI.
ClienteUiState.Fase.Error	Presentación	Representa fallo con mensaje.	Ejecutar la carga o el reintento.	No puede representarse esta situación de UI.
ClienteScreen (función)	Presentación	Observa el estado y delega eventos al ViewModel.	Validar negocio o crear repositorios.	No se puede operar visualmente este módulo.
Screen	Presentación	Define los destinos de navegación.	Aplicar reglas o persistir datos.	El enrutado deja de ser exhaustivo.
Screen.Inicio	Presentación	Identifica el destino Inicio.	Construir repositorios.	Falta el destino en la navegación.
Screen.Productos	Presentación	Identifica el destino Productos.	Construir repositorios.	Falta el destino en la navegación.
Screen.Clientes	Presentación	Identifica el destino Clientes.	Construir repositorios.	Falta el destino en la navegación.
Screen.Pedidos	Presentación	Identifica el destino Pedidos.	Construir repositorios.	Falta el destino en la navegación.
Destino (privada, App.kt)	Presentación	Relaciona Screen, título e icono del menú.	Ejecutar casos de uso.	Se pierde la estructura común de destinos.
App (función)	Presentación	Compone tema, menú y pantalla seleccionada.	Guardar entidades o validar negocio.	No hay raíz compartida de interfaz.
InicioScreen (función)	Presentación	Presenta portada y accesos de la aplicación.	Consultar datos directamente.	Falta la pantalla de bienvenida.
PharmaMobilTheme (función)	Presentación	Aplica MaterialTheme según modo claro u oscuro.	Contener reglas de negocio.	Se pierde el tema central de la app.

Clase / componente	Capa	Responsabilidad en una frase	Lo que tiene prohibido hacer	Qué se rompería si desapareciera
ValidatedTextField (función)	Presentación	Muestra entrada, ayuda y error proporcionados.	Decidir si un producto es válido.	Se duplica la presentación de campos y errores.
MensajeExito (función)	Presentación	Renderiza una confirmación de operación.	Registrar productos o decidir éxito.	Falta la confirmación visual reutilizable.
EstadoVacio (función)	Presentación	Muestra icono, texto y acción opcional.	Cargar inventario por cuenta propia.	Falta la vista común de vacío, error y pendientes.
AppModule.kt (archivo)	Composición	Declara módulos y arranca Koin con initKoin.	Ser dependencia de los modelos.	No se ensambla el grafo de dependencias.
dataModule (valor)	Composición	Vincula contratos a repositorios singleton.	Ejecutar reglas del formulario.	Koin no resuelve repositorios por interfaz.
domainModule (valor)	Composición	Registra los cuatro casos de uso como factory.	Contener reglas de validación.	Koin no construye los casos de uso.
presentationModule (valor)	Composición	Registra los dos ViewModels con sus casos de uso.	Renderizar pantallas.	La UI no obtiene ViewModels por Koin.
platformModule (expect/actual)	Composición	Reserva dependencias de Android e iOS en sus source sets.	Introducir Android o iOS en dominio.	Se pierde el punto específico de plataforma.
initKoinIos (función, iosMain)	Composición	Expone el arranque de Koin al código Swift.	Duplicar el grafo común.	El arranque Swift pierde su función de inicialización.
MainViewController (función, iosMain)	Presentación	Aloja App en un Compose UIViewController.	Aplicar validación o acceder a datos.	SwiftUI pierde la entrada a la UI compartida.

Clases de pruebas del módulo shared

Clase	Capa	Responsabilidad en una frase	Lo que tiene prohibido hacer	Qué se rompería si desapareciera
ProductoFlujoTest	Pruebas	Comprueba los cuatro escenarios exactos del Reto 02.	Participar en el código de producción.	Falta evidencia de flujo exigida por la guía.
ProductoFlujoTest.RepositorioControlado	Pruebas	Fuerza listas o excepción y cuenta invocaciones.	Participar en el código de producción.	No se demuestra ausencia de efectos con precio cero.
FakeProductoRepository	Pruebas	Ofrece inventario y fallos controlables para pruebas previas.	Participar en el código de producción.	Pruebas de producto pierden su doble.
FakeClienteRepository	Pruebas	Ofrece clientes y fallos controlables.	Participar en el código de producción.	Pruebas de cliente pierden su doble.
ProductoViewModelTest	Pruebas	Comprueba formato, validación, carga y registro de productos.	Participar en el código de producción.	Se pierde cobertura previa de presentación.

Clase	Capa	Responsabilidad en una frase	Lo que tiene prohibido hacer	Qué se rompería si desapareciera
ClienteViewModelTest	Pruebas	Comprueba estados y acciones de clientes.	Participar en el código de producción.	Se pierde cobertura de la presentación de clientes.
RegistrarProductoUseCaseTest	Pruebas	Verifica validación y registro de productos.	Participar en el código de producción.	Se debilita la detección de reglas incorrectas.
RegistrarClienteUseCaseTest	Pruebas	Verifica validación y registro de clientes.	Participar en el código de producción.	Se debilita la detección de datos de contacto inválidos.
ListarProductosUseCaseTest	Pruebas	Verifica listado correcto y fallo encapsulado.	Participar en el código de producción.	No se detectan regresiones de consulta.
ProductoTest	Pruebas	Verifica invariantes y reposición del producto.	Participar en el código de producción.	Se pierde cobertura del modelo Producto.
ClienteTest	Pruebas	Verifica invariantes del cliente.	Participar en el código de producción.	Se pierde cobertura del modelo Cliente.
ProductoRepositorioEnMemoriaTest	Pruebas	Verifica almacenamiento e ids de productos.	Participar en el código de producción.	No se detectan regresiones del repositorio.
ClienteRepositorioEnMemoriaTest	Pruebas	Verifica almacenamiento e ids de clientes.	Participar en el código de producción.	No se detectan regresiones de persistencia temporal.
AppModuleTest	Pruebas	Comprueba resolución del grafo Koin.	Participar en el código de producción.	Errores de ensamblaje llegarían a ejecución de la app.
SharedCommonTest	Pruebas	Comprueba una aserción de ejemplo en commonTest.	Participar en el código de producción.	Se pierde solo la prueba inicial, no cobertura de negocio.
SharedLogicAndroidHostTest	Pruebas	Comprueba una aserción básica en androidHostTest.	Participar en el código de producción.	Falta el ejemplo específico del host Android.
SharedLogicIOSTest	Pruebas	Comprueba una aserción básica en iosTest.	Participar en el código de producción.	Falta el ejemplo específico del target iOS.

Los companion object guardan constantes: Producto.STOCK_MINIMO y retardos de los repositorios. Los helpers privados de pantallas y los mapeadores aUi() pertenecen a presentación; no son nuevas capas ni clases. Las pruebas de ejemplo no acreditan flujos funcionales.

3. Decisiones justificadas

Modelé ProductoUiState como una data class que conserva formulario, indicador de registro y mensaje de éxito, junto con una Fase sellada para el inventario. Así puedo representar Cargando, SinProductos, ConProductos o Error de forma excluyente sin borrar lo que la persona escribió. Frente a varios booleanos independientes, evito combinaciones contradictorias, como cargar y mostrar un error terminal simultáneamente. Frente a sellar todo el estado, evito repetir el formulario en cada variante o perderlo al cambiar de fase. El when exhaustivo obliga a atender nuevas fases; StateFlow expone una instantánea de solo lectura. El indicador registrando describe una operación distinta del listado y por eso permanece separado.

Trasladé las reglas de ProductoValidator a RegistrarProductoUseCase porque un registro debe ser válido aunque llegue desde otra pantalla. El caso de uso valida antes de llamar a ProductoRepository y devuelve errores por campo dentro de ProductoInvalidoException. Producto también protege invariantes cuando se construye directamente: esa defensa no sustituye los mensajes útiles del formulario. Mantener reglas únicamente en Compose habría permitido saltarlas desde otro consumidor. Respecto de ClienteValidator, la guía describe una versión anterior: en el código revisado ya no existe y RegistrarClienteUseCase concentra nombre, correo y teléfono. Mantendría esa migración y reservaría a la presentación el formato, los mensajes y los eventos; no reintroduciría el validador de negocio en la pantalla.

Cuando exista la implementación remota, cambiaría el enlace de la línea 21 de shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/di/AppModule.kt: single<ProductoRepository> { ProductoRepositorioEnMemoria() }. Lo sustituiría por una construcción de ProductoRepositorioRemoto con sus dependencias reales. La línea 9, que importa la implementación en memoria, también se actualizaría. La sustitución del enlace es puntual, pero desarrollar el cliente HTTP, los mapeos y su configuración no equivale a cambiar una sola línea. Los casos de uso y ProductoViewModel seguirían usando el mismo contrato suspend; las dependencias específicas del motor se registrarían en platformModule. Esta separación permite conservar las pruebas con repositorios falsos mientras se incorpora el backend.

Extensión de la justificación: 316 palabras. Líneas verificadas sobre la versión local documentada.

4. Pruebas de flujo

Se agregó ProductoFlujoTest.kt en shared/src/commonTest/kotlin/pe/edu/upeu/pharmamobil/presentation/producto. Sus cuatro pruebas usan RepositorioControlado, una implementación falsa de ProductoRepository con las dos funciones suspend del contrato. No requieren emulador, red ni el repositorio de producción.

Cada prueba ejecuta runTest, instala StandardTestDispatcher(testScheduler) como Dispatchers.Main y avanza con advanceUntilIdle() antes de leer el resultado. Los tres casos de carga también comprueban Cargando antes de avanzar. finally restaura Dispatchers.Main para no contaminar otras pruebas.

Matriz de pruebas de flujo

N.º / condición inicial forzada	Acción ejecutada	Estado esperado	Estado observado	Resultado
1. Repositorio falso vacío	Crear ViewModel y avanzar dispatcher.	Fase.SinProductos	Android: Cargando → SinProductos. iOS: no ejecutado.	Android: PASS. iOS: pendiente.
2. Repositorio falso con tres productos	Crear ViewModel y avanzar dispatcher.	Fase.ConProductos con 3 elementos	Android: Cargando → ConProductos; lista mapeada completa. iOS: no ejecutado.	Android: PASS. iOS: pendiente.
3. Repositorio falso lanza excepción	Crear ViewModel y avanzar dispatcher.	Fase.Error	Android: Error con “Sin conexión”. iOS: no ejecutado.	Android: PASS. iOS: pendiente.
4. Nombre Paracetamol, precio “0”, stock “5”	Completar carga inicial; llenar formulario; registrar y avanzar.	Error de precio sin llamar al repositorio durante el registro	Android: “El precio debe ser mayor a 0”; 0 registros; sin listados adicionales. iOS: no ejecutado.	Android: PASS. iOS: pendiente.

En el caso 4 ya ocurrió un listado al construir el ViewModel. “Sin llamar al repositorio” se verifica sobre la acción de registrar: cero llamadas a registrar y ninguna llamada adicional a listar. También se comprueba que registrando vuelve a false, el precio “0” permanece, no hay éxito y la fase sigue en SinProductos.

Resultados reproducibles

Android: gradlew.bat :shared:testAndroidHostTest --rerun-tasks --console=plain. BUILD SUCCESSFUL; 53 pruebas, 0 fallos, 0 errores y 0 omitidas. ProductoFlujoTest: 4 pruebas satisfactorias. Log: docs/evidencias/android-host-test.txt. Resultados originales: shared/build/test-results/testAndroidHostTest. Informe: shared/build/reports/tests/testAndroidHostTest/index.html.

iOS: gradlew.bat :shared:iosSimulatorArm64Test --console=plain. La compilación de pruebas concluyó, pero linkDebugTestIosSimulatorArm64 e iosSimulatorArm64Test aparecen como SKIPPED. BUILD SUCCESSFUL no significa que las pruebas iOS hayan pasado. Log: docs/evidencias/ios-simulator-test.txt.

Evidencias por prueba: Android

Abrir en Android Studio el archivo ProductoFlujoTest.kt. Ejecutar la clase y expandir sus cuatro resultados. Alternativamente, abrir el informe HTML indicado y seleccionar la clase. Cada captura debe mostrar nombre del caso, resultado y contexto de ejecución; insertar imágenes reales en estas cajas.

Captura 1 · caso 1

SE NECESITA CAPTURA REAL
En Android Studio o en el reporte HTML, localizar repositorioVacioProduceSinProductos. Mostrar la prueba aprobada y las aserciones de Cargando y SinProductos.

Captura 2 · caso 2

SE NECESITA CAPTURA REAL
En Android Studio o en el reporte HTML, localizar tresProductosProducenConProductos. Mostrar la prueba aprobada y la comprobación de los tres productos.

Captura 3 · caso 3

SE NECESITA CAPTURA REAL
En Android Studio o en el reporte HTML, localizar excepcionProduceError. Mostrar la prueba aprobada y el mensaje “Sin conexión”.

Captura 4 · caso 4

SE NECESITA CAPTURA REAL
En Android Studio o en el reporte HTML, localizar precioCeroDejaErrorSinLlamarAlRepositorio. Mostrar la prueba aprobada, el error de precio, registros = 0 y listados sin incremento.

Evidencias de ejecución por plataforma

SE NECESITA CAPTURA REAL
Android: desde la raíz del proyecto, ejecutar .\gradlew.bat :shared:testAndroidHostTest --rerun-tasks --console=plain. Capturar la terminal con el comando y BUILD SUCCESSFUL; acompañar con el resumen del informe HTML que muestre 53 pruebas y cero fallos.

SE NECESITA CAPTURA REAL
iOS: en una Mac con Xcode y simulador iOS ARM64 disponible, abrir el mismo proyecto y ejecutar ./gradlew :shared:iosSimulatorArm64Test --rerun-tasks --console=plain. Capturar el comando y las pruebas ejecutadas (no SKIPPED). Mostrar los cuatro casos de ProductoFlujoTest y completar la columna iOS de la matriz con el resultado real.

Captura opcional de la aplicación

SE NECESITA CAPTURA REAL
Abrir PharmaMobil en Android Studio, ejecutar androidApp en un emulador o teléfono y entrar a Productos. Escribir Paracetamol, precio 0 y stock 5; pulsar Registrar. Capturar el formulario con “El precio debe ser mayor a 0”. Esta imagen complementa las pruebas; por sí sola no demuestra que el repositorio no recibió llamadas.

Para completar la evidencia iOS se necesita ejecución en macOS. No se adjuntan imágenes simuladas ni se marca como aprobada una tarea omitida. Los registros de texto y XML incluidos son evidencia real de la ejecución Android realizada durante esta actividad.

5. Bitácora de dificultades

Problema encontrado	Cómo se abordó	Resultado / verificación
La cobertura existente no coincidía con la matriz: un producto y precio “abc”.	Se añadió ProductoFlujoTest con tres productos y precio “0”, más un falso con contadores.	Cuatro pruebas nuevas pasan; se verifica que validar precio no produce efectos en el repositorio.
El ViewModel inicia corutinas; leer inmediatamente podía observar la fase inicial.	Se compartió testScheduler con StandardTestDispatcher y se usó advanceUntilIdle.	Los casos de carga verifican fase inicial y final de forma determinista.
La guía y el tutorial histórico mencionan ClienteValidator, ausente en el código actual.	Se inspeccionó RegistrarClienteUseCase y el estado de clientes. Se documentó la migración existente.	La tabla y la justificación describen las clases reales, sin inventar un validador vigente.
La primera ejecución de Gradle no pudo descargar la distribución por restricciones de red.	Se repitió la ejecución con acceso autorizado y el JDK incluido en Android Studio.	BUILD SUCCESSFUL; 53 pruebas Android sin fallos.
La tarea de iOS termina como SKIPPED en Windows.	Se registró la salida y se dejó el procedimiento de ejecución en Mac con Xcode.	Compilación completada; ejecución iOS pendiente, sin atribuirle un PASS.

Comprobación final frente a la guía

Se incluyen portada, índice, repositorio, cinco secciones, diagrama de elaboración propia, tabla con las cinco columnas, justificación de 250 a 400 palabras, punto de cambio con archivo y línea, matriz y bitácora. Las cuatro pruebas nuevas están en commonTest y pasan en Android. Faltan las capturas reales, la ejecución iOS, completar los datos del estudiante y publicar la rama antes de entregar en el aula virtual.

Preparación del envío

Completar nombres, código y fecha de entrega; insertar las capturas indicadas y actualizar iOS con resultados comprobados. Revisar y publicar la rama codex/actividad-autonoma-s5 en el repositorio. Finalmente, subir Manual-Arquitectura-PharmaMobil_Acevedo.pdf al aula virtual. La creación de este documento no realiza una entrega académica ni confirma la publicación remota.

Fuentes y referencias (APA 7)

Actividad autónoma: Sesión 5, Unidad 1. Manual de arquitectura de PharmaMobil: responsabilidades por capa y pruebas de flujo. (s. f.). [Guía de actividad no publicada, proporcionada por el estudiante].

miguelon05. (s. f.). PharmaMobile [Código fuente]. GitHub. https://github.com/miguelon05/PharmaMobile. Código local base 1565816 y pruebas añadidas en esta actividad.

Las figuras son de elaboración propia. El código y los registros locales constituyen la fuente primaria de las responsabilidades y los resultados; no se han usado capturas ajenas como evidencia.